

VORTEX-GEN ARCHITECTURE

1. Executive Summary & Infrastructure

System Overview

Vortex-Gen is a full-stack, distributed application designed for aerospace engineering. It is composed of a modern React 19 Frontend and a hybrid Node.js/Python Backend. The architecture is explicitly designed for high-performance computing, minimizing the latency between the browser-based 3D simulation and the intensive PyTorch neural network inferences using a persistent IPC Daemon.

Infrastructure & Hosting

- Frontend UI: React 19, Vite, Tailwind v4, Zustand. Hosted on Vercel.
- Backend API: Node.js (Express) acting as a gateway and Auth perimeter. Hosted on Hugging Face.
- ML Engine: Python 3, PyTorch (NeuralFoil daemon) for millisecond CFD inferences.
- Database & Auth: Supabase (PostgreSQL, Row-Level Security, JWT Auth).
- Payments & Email: Stripe for auto-tier upgrades via Webhooks, Resend for dynamic HTML emails.

VORTEX-GEN ARCHITECTURE

2. Core Features & Data Flow

Feature-Based Frontend Architecture

The frontend leverages a domain-driven structure mapping UI, logic, and state into distinct feature folders:

- Academy & Lab: Educational walkthroughs and the core 3D simulation engine.
- Compare Mode: Simultaneous dual-geometry rendering with overlaid Cl/Cd analytics.
- Artifact Generation: Client-side extraction of 3D-printable STLs and multi-page analytical PDFs using jsPDF.

Data Flow & Security

- 1. Client triggers simulation with shape geometry (points) and environment config.
- 2. React sends POST /api/analyze to Express backend with Supabase JWT.
- 3. Express middleware validates JWT against Supabase. Checks subscription tier.
- 4. Express converts request to JSON and writes to Python NeuralFoil daemon's stdin.
- 5. Python process decodes JSON, runs PyTorch NeuralFoil surrogate.
- 6. Python writes results array back to stdout as JSON.
- 7. Express parses stdout, streams response back to React frontend.
- 8. Recharts & Three.js consume data for Cl/Cd charts and 3D particle visualization.

VORTEX-GEN ARCHITECTURE

3. Directory Structure

```
backend/
|-- run_nf.py           # Python NeuralFoil daemon (stdin/stdout protocol)
|-- server.js           # Express API, Stripe Webhooks & JWT Auth
|-- requirements.txt    # Python dependencies
+-- package.json        # Node dependencies

frontend/
|-- public/             # Static assets
+-- src/
    |-- assets/          # SVGs and UI graphics
    |-- components/      # Shared global UI components
    |-- features/        # Feature-Sliced Architecture Domains
    |   |-- academy/     # Educational learning modules
    |   |-- lab/         # Aerodynamic engine, Three.js canvas, Telemetry
    |   |-- pricing/     # Stripe subscription models
    |   |-- profile/     # User workspaces and settings
    |-- lib/             # Supabase SDK clients
    |-- pages/           # Routing definitions
    |-- store/           # Zustand global state
    |-- App.jsx / App.tsx # Root React entry
    +-- index.css        # Tailwind v4 directives
```